

Video 002 — Building REST APIs and Database Details in Spring Boot

Video info

Field	Detail
Playlist	Chai aur Spring Boot
Position	Video 2 of 4
Topic	REST API fundamentals, building full CRUD REST APIs (in-memory, then a real database), the JDBC→Hibernate→JPA→Spring Data JPA evolution, entity relationships, the persistence context, query methods, pagination
Instructor	Aniket (Chai aur Code)
Duration	3h 40m 6s
Watch	https://www.youtube.com/watch?v=yDF3ZOp0uzQ
Playlist	https://www.youtube.com/playlist?list=PLAtsKd9vhf5M

How to read these notes. Same conventions as [video 1's notes](#):

📌 **Beyond the lecture** adds context not in the video but needed in practice. ⚠️ **Nuance** flags a spot where the lecture simplifies something. 🟡 **Coming from JS?** maps the concept onto Node/Express (and its ORMs — Prisma, Sequelize, TypeORM — since this video is heavily about the database layer). This video is long and dense; these notes follow it section by section so nothing taught gets dropped, then add what a production backend needs on top.

What this lecture covers

Two halves, roughly split at the 1:04:00 mark:

Part 1 — REST APIs (in-memory, no database yet):

1. What an API is, what makes an API "REST"
2. HTTP request/response structure, methods, status codes
3. Path variables vs. query parameters
4. Building a full CRUD REST API backed by an in-memory list — Controller → Service → Repository layering, DTOs

Part 2 — Connecting to a real database:

1. Why JDBC → Hibernate → JPA → Spring Data JPA evolved, in that order
2. Setting up Postgres via Docker + pgAdmin, and Spring Boot's datasource config
3. `@Entity` classes, primary keys, table/relationship annotations

4. `JpaRepository`, the entity lifecycle, the persistence context, and "dirty checking"
5. Rebuilding the full CRUD API against the real database, plus a nested `/users/{id}/orders` resource
6. Spring Data JPA query techniques: derived query methods, JPQL, native queries, pagination and sorting

Part 1: REST API fundamentals

02:55 — What is an API?

API stands for Application Programming Interface — a contract that allows software systems to communicate.

The analogy used: a restaurant. You (the **client**) don't walk into the kitchen (the **server**) yourself — you tell a **waiter** what you want, the waiter carries that to the kitchen, and carries the finished dish back to you. The waiter is the API: the thing standing between client and server that carries requests one way and data back the other way.

REST is one style of API (others: **gRPC**, **GraphQL** — named but not covered here).

05:39 — What makes an API a REST API

REST = **R**epresentational **S**tate **T**ransfer. Practically: an API that uses HTTP/HTTPS, URLs, and JSON responses to let a client and server communicate.

Properties named in the video:

- **Resource-based** — everything revolves around nameable resources (users, orders)
- **Stateless** — no session state kept on the server between requests
- **Uses standard HTTP methods** — GET, POST, PUT, DELETE, PATCH
- **JSON representation** — data returned as JSON

⚠ **Nuance — this is "pragmatic REST," not the formal definition.**

The video's description is exactly what the industry means by "REST API" day to day, and it's the right mental model to build APIs with. But the term originates from Roy Fielding's 2000 dissertation, which defines REST more strictly — including **HATEOAS** (Hypermedia as the Engine of Application State: responses should include links to related actions/ resources, so a client can navigate the API without hardcoding URLs). Almost no API anyone calls "a REST API" today implements HATEOAS. What the video teaches — and what you'll build professionally — is more accurately "a JSON-over-HTTP API following REST conventions." Nobody will blink if you call it a REST API; just know the term is used loosely industry-wide, not only in this video.

06:23 — Resources

`/users` is a **collection resource** (a list); `/users/1` is a **single resource** (identified by its path). This distinction drives which HTTP verb is used where (see below).

07:00 — HTTP request structure

Part	Purpose	Example
Method	What operation to perform	<code>POST</code>
URL	Which resource	<code>/users</code>
Headers	Metadata about the request	<code>Content-Type: application/json</code>
Body	The actual payload	<code>{"name": "...", "email": "..."} </code>

08:23 — HTTP response structure

Part	Purpose	Example
Status code	Outcome of the request	<code>200</code>
Headers	Metadata about the response	—
Body	The returned data	<code>{"id": 1, "name": "..."} </code>

08:57 — HTTP methods and resource naming

Method	Purpose	Example
<code>GET</code>	Retrieve data	<code>GET /users</code> (collection), <code>GET /users/5</code> (single)
<code>POST</code>	Create a new resource	<code>POST /users</code>
<code>PUT</code>	Replace/update a whole resource	<code>PUT /users/5</code>
<code>PATCH</code>	Update part of a resource	<code>PATCH /users/5</code>
<code>DELETE</code>	Remove a resource	<code>DELETE /users/5</code>

The video is explicit about a naming convention worth internalizing: the resource segment is always **plural** (`/users`, not `/user`) — even when a single resource is being addressed by ID (`/users/5`), because the resource type itself (users) is what's plural, not the specific item.

10:11 — Path variables vs. query parameters

- **Path variable** — identifies a specific resource, part of the URL path

itself: `/users/5` (the `5` is a path variable).

- **Query parameter** — used for filtering, searching, or pagination,

appended after `?`: `/users?age=25&page=1&size=10`.

 **Coming from JS?** Nothing here differs from Express — `req.params.id` for path variables, `req.query.age` for query parameters. Spring just extracts these into method parameters (`@PathVariable`, `@RequestParam`) instead of handing you one `req` object to destructure yourself.

Part 2: Building a REST API — in-memory first

A new project (`rest-demo` , Maven, Java, Spring Web dependency) is used to build a full CRUD API for a `User` resource **before** any database is connected — deliberately, so the REST layer can be learned in isolation from persistence concerns.

13:08 — Package layout

```
com.example.restdemo
├─ controller/      → handles HTTP requests
├─ dto/             → Data Transfer Objects (what the API sends/receives)
├─ repository/      → holds/returns data (in-memory list, for now)
└─ service/         → business logic, sits between controller and repository
```

13:19 — DTO vs. Entity, named explicitly for the first time

DTO = **Data Transfer Object** — the object used to transfer data to/from the API. **Entity** is the separate term used later for a class mapped to a database table (e.g. a `User` or `Student` table).

```
public class UserDTO {
    private Long id;
    private String name;
    private String email;

    public UserDTO(Long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    // getters and setters — Spring Boot uses these to convert to/from JSON
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

The video is explicit about *why* getters/setters matter even with private fields: Jackson (Spring Boot's default JSON converter) reads and writes object state through them.

17:01 — Repository holding in-memory data (a stand-in for the database)

```
@Repository
public class UserRepository {
    private final List<UserDTO> users = new ArrayList<>();

    public UserRepository() {
        users.add(new UserDTO(1L, "Aniket", "aniket@test.com"));
        users.add(new UserDTO(2L, "Ankit", "ankit@test.com"));
        users.add(new UserDTO(3L, "Nikita", "nikita@test.com"));
    }

    public List<UserDTO> findAll() {
        return users;
    }
}
```

The video is explicit that this is a deliberate stand-in — *****assume this data is coming from a database***** — purely so a proper Controller → Service → Repository architecture can be practiced before the real database is wired in later in the same video. `@Repository` here is just the stereotype annotation from video 1 — this class becomes a Spring-managed singleton bean either way.

19:59 — Service layer

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<UserDTO> getAllUsers() {
        return userRepository.findAll();
    }
}
```

Constructor injection again (from video 1) — no `@Autowired` needed.

21:53 — Controller: `@RequestMapping` as a shared base path

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public List<UserDTO> getAllUsers() {
        return userService.getAllUsers();
    }
}
```

Class-level `@RequestMapping("/api/users")` sets a base path so every method below doesn't have to repeat `/api/users` — it only adds its own suffix (or nothing, for the collection root). The video demonstrates this concretely by temporarily removing it: every endpoint 404s because Spring no longer knows the base path, and it has to be re-added to *every single* `@GetMapping` individually.

27:37 — Get user by ID: `@PathVariable`

```
@GetMapping("/{id}")
public UserDTO getUserById(@PathVariable Long id) {
    return userService.getUserById(id);
}
```

```
// UserService
public UserDTO getUserById(Long id) {
    for (UserDTO user : userRepository.findAll()) {
        if (user.getId().equals(id)) {
            return user;
        }
    }
    return null;
}
```

The video is explicit that returning `null` on a miss, and using a plain linear loop over `Optional / Stream`, is a deliberate simplification — "we'll use `Optional` once real data is coming from the database... keeping things simple right now because our goal is to make the REST API work."

35:41 — HTTP status codes reference

The video walks `HttpStatus` in the IDE and highlights the ones you'll actually reach for:

Code	Meaning	When
<code>200 OK</code>	Success	Default for a successful GET
<code>201 Created</code>	A resource was created	Response to a successful POST
<code>204 No Content</code>	Success, nothing to return	Response to a successful DELETE
<code>400 Bad Request</code>	Client sent invalid input	Failed validation
<code>401 Unauthorized</code>	Not authenticated	Missing/invalid credentials
<code>403 Forbidden</code>	Authenticated but not permitted	Lacking required permissions
<code>404 Not Found</code>	Resource doesn't exist	e.g. <code>GET /users/999</code> when 999 doesn't exist
<code>429 Too Many Requests</code>	Rate limit exceeded	—
<code>500 Internal Server Error</code>	Server-side failure	Unhandled exception
<code>502 Bad Gateway</code>	Upstream failure	—

37:01 — `ResponseEntity<T>`: controlling status code and body explicitly

```
@GetMapping("/{id}")
public ResponseEntity<UserDTO> getUserById(@PathVariable Long id) {
    UserDTO user = userService.getUserById(id);
    return ResponseEntity.status(HttpStatus.OK).body(user);
    // equivalently: return ResponseEntity.ok(user);
}
```

Returning a plain `UserDTO` implicitly returns `200 OK`. `ResponseEntity` is needed the moment you want a *different* status code — e.g. `201 Created` after a POST, `204 No Content` after a DELETE.

 **Coming from JS?** `ResponseEntity.status(HttpStatus.CREATED).body(data)` is doing exactly what `res.status(201).json(data)` does in Express — one object bundles status code + body, same as Express's chained call.

40:01 — Create user (POST): a separate DTO for the request body

The video makes a deliberate design point here: the DTO you *receive* on create shouldn't be the same shape as the one you *return*, because the client shouldn't be supplying the `id` — the server generates it.

```
public class CreateUserDTO {
    private String name;
    private String email;
    // constructor, getters, setters — no id field
}
```

```
@PostMapping
public ResponseEntity<UserDTO> createUser(@RequestBody CreateUserDTO createUserDTO) {
    return
    ResponseEntity.status(HttpStatus.CREATED).body(userService.createUser(createUserDTO));
}
```

```
// UserService
public UserDTO createUser(CreateUserDTO createUserDTO) {
    UserDTO newUser = new UserDTO(
        UUID.randomUUID().toString(), // see nuance below re: the ID type
        createUserDTO.getName(),
        createUserDTO.getEmail()
    );
    return userRepository.save(newUser);
}
```

```
// UserRepository
public UserDTO save(UserDTO user) {
    users.add(user);
    return user;
}
```

⚠️ ****Nuance — the ID type change from `Long` to `String` is scoped to this in-memory demo, not a general recommendation.**** Partway through, the video changes `UserDTO.id` from `Long` to `String` specifically so `UUID.randomUUID().toString()` can be used to generate IDs without a real auto-incrementing database column to lean on. Once the real database is wired up later in this same video, IDs go back to `Long` with `@GeneratedValue(strategy = GenerationType.IDENTITY)` (auto-increment) — because that's what a real relational primary key naturally gives you for free. ****Auto-increment `Long` vs. `UUID` id is a genuine, separate architectural decision**** you'll actually face (see the Beyond the Lecture note under Entities below) — it isn't something the video is teaching as a tradeoff here, just a means to an end for the in-memory demo.

51:39 — Update user (PUT): full replace

```
@PutMapping("/{id}")
public ResponseEntity<UserDTO> updateUser(@PathVariable Long id, @RequestBody UpdateUserDTO
updateUserDTO) {
    return ResponseEntity.status(HttpStatus.OK).body(userService.updateUser(updateUserDTO,
id));
}
```

```
// UserService — simplified, no real validation yet (flagged explicitly by the video)
public UserDTO updateUser(UpdateUserDTO updateUserDTO, Long id) {
    for (UserDTO user : userRepository.findAll()) {
        if (user.getId().equals(id)) {
            user.setName(updateUserDTO.getName());
            user.setEmail(updateUserDTO.getEmail());
            return user;
        }
    }
    return null;
}
```

The video explicitly flags this as intentionally unfinished: *"when we do it with a proper database, we'll also do proper validations there, where we'll throw the error... let's not overcomplicate things, focus is on REST APIs right now."* No 404 is returned for a missing user yet — that's deferred.

59:20 — Delete user

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
    userService.deleteUserById(id);
    return ResponseEntity.noContent().build(); // 204
}
```

```
// UserRepository
public void deleteUserById(Long id) {
    users.removeIf(user -> user.getId().equals(id));
}
```

By this point all five operations exist: ***Get All, Get By ID, Create, Update, Delete*** — all against the in-memory list. The video's summary line: *"Patch we'll create once we connect the database"* — deferred to Part 2.

Part 3: Why did Spring Data JPA need to exist?

Not code yet — the conceptual foundation before touching a real database. Four layers evolved in this order, **each one fixing the previous layer's problem, not replacing it wholesale**:

```
JDBC → Hibernate → JPA → Spring Data JPA
```

1:06:00 — JDBC (Java Database Connectivity)

Problem it solved: every database vendor (Oracle, MySQL, Postgres) had its own proprietary API — no standard way to talk to *any* database from Java.

```
Connection conn = DriverManager.getConnection(url, user, password);
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE id = ?");
stmt.setLong(1, id);
ResultSet rs = stmt.executeQuery();
if (rs.next()) {
    User user = new User();
    user.setName(rs.getString("name")); // manual mapping, every field, every query
}
```

What JDBC left unsolved:

- No object mapping — you manually pull each column into each field, every time
- Heavy boilerplate — connection handling, statement handling, exception handling, repeated in every method
- Tight coupling to raw SQL scattered everywhere — a schema change means hunting down and fixing every query

1:09:21 — Hibernate (an ORM — Object-Relational Mapper)

Problem it solved: the object-mapping and boilerplate pain from JDBC. Hibernate maps Java classes ("**entities**") directly to database tables, so:

```
userRepository.save(user); // Hibernate generates and runs the INSERT/UPDATE SQL for you
```

No manual SQL, no manual field-by-field mapping, and it handles relationships (one-to-many, etc.) for you.

What Hibernate left unsolved: it wasn't the *only* ORM — EclipseLink and OpenJPA existed too, each with its own, incompatible API. Switching ORMs meant rewriting everything — **no portability, no standard**.

1:12:41 — JPA (Java Persistence API)

Problem it solved: standardization. JPA is a **specification**, not an implementation — Hibernate, EclipseLink, etc. all implement the *same* JPA API (`jakarta.persistence.*`), so:

- Common API (`EntityManager.persist(...)`) works regardless of which ORM implements it underneath

- You can switch ORMs (in theory) without rewriting application code
- Standard annotations: `@Entity`, `@Id`, `@OneToMany`, etc.
- **JPQL** — its own query language, operating on entities/fields rather than tables/columns (more below)

What JPA left unsolved: still a lot of boilerplate — manually creating and managing an `EntityManager`, a separate DAO ("Data Access Object") layer per entity, manual transaction handling. Repetitive CRUD methods still had to be hand-written for every entity.

1:15:07 — Spring Data JPA

Problem it solved: the remaining boilerplate. You declare an interface; Spring Data JPA generates the implementation:

```
public interface UserRepository extends JpaRepository<User, Long> {  
    // save(), findAll(), findById(), deleteById() — all free, zero implementation  
}
```

Plus: query generation from method names alone (`findByName(String name)`), built-in pagination/sorting support, and tight integration with Spring's dependency injection and transaction management.

1:18:04 — The one-line summary

Layer	Fixed	Left behind
JDBC	Standard way to connect to any DB	No object mapping, heavy boilerplate, SQL scattered everywhere
Hibernate	Object-relational mapping	No standard across different ORMs
JPA	Standardized ORM API	Still lots of manual boilerplate (EntityManager, DAOs)
Spring Data JPA	Removes the remaining boilerplate	<i>(this is where you'll actually work day to day)</i>

Coming from JS? — this evolution has a near-exact Node parallel.

Node's database-access ecosystem evolved through a strikingly similar arc:

Java world	Node.js world
Raw JDBC	Raw drivers: <code>pg</code> , <code>mysql2</code> , <code>mongodb</code> — you write every SQL string by hand
Hibernate (an ORM, one of several)	Sequelize / Mongoose — an ORM/ODM, one of several, each with its own API
JPA (a <i>standard</i> ORM spec, multiple implementations)	<i>(no true equivalent — JS has no vendor-neutral persistence standard; each ORM is a closed ecosystem)</i>
Spring Data JPA (repository interfaces, zero boilerplate)	Prisma — schema-first, generates a fully-typed client, closest in spirit to "here's the model, give me CRUD for free"

Worth calling out directly: **TypeORM's decorators (`@Entity()`, `@Column()`, `@ManyToOne()`, `@PrimaryGeneratedColumn()`) are a deliberate, close port of JPA/Hibernate's annotations to TypeScript** — if you've used TypeORM, the `@Entity` / `@Id` / `@ManyToOne` syntax you're about to see below will look almost identical to code you've already written.

Part 4: Setting up a real database

1:18:52 — New project

New project (`spring-data-jpa-demo` , Maven, Java 17), with **three** dependencies this time: `Spring Web` , `Spring Data JPA` , and — new — the **PostgreSQL Driver** (`org.postgresql:postgresql`). The video is explicit that Spring Data JPA alone isn't enough: you still need the vendor-specific JDBC driver underneath it to actually open a socket to Postgres.

1:21:34 — Running Postgres via Docker

```
docker run --name postgres-container \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=testdb \
  -p 5432:5432 \
  -d \
  postgres
```

The video is explicit that Docker is used here specifically *because* it'll be reused throughout the rest of the series for Redis, Kafka, etc. (video 4) — getting comfortable with `docker run` now pays off later.

1:25:46 — pgAdmin (a GUI for inspecting the database)

A separate download, used to register a server connection (host `localhost`, port `5432`, the `postgres` / `postgres` credentials from the Docker command above) and visually browse databases, tables, and run queries — an alternative to typing raw SQL in a terminal.

1:30:03 — `application.properties` : the datasource config

```
spring.datasource.url=jdbc:postgresql://localhost:5432/testdb
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

1:31:26 — `ddl-auto` — every mode, and which ones never touch production

Value	Behavior	Use
<code>create</code>	Drops and recreates the whole schema every app start	Never — destroys data on every restart
<code>create-drop</code>	Creates on start, drops everything on shutdown	Ephemeral testing only
<code>update</code>	Adds new columns/tables for schema changes; never deletes	Local development/testing convenience
<code>validate</code>	Checks entity definitions match the actual DB schema; fails startup on mismatch	Production
<code>none</code>	Hibernate touches nothing; all schema changes are manual	Production, paired with a migration tool

The video repeats, more than once and with emphasis: ****never use `create`, `create-drop`, or `update` in production.**** `validate` is the production choice — it protects the database from silent, accidental tampering by failing loudly if code and schema disagree.

 **Beyond the lecture — the video names Flyway once, in passing, and moves on. Here's what actually fills the `none` / `validate` gap.**

If Hibernate isn't allowed to touch your schema in production, *something* has to apply changes deliberately and safely. That's what a migration tool does: **Flyway** or **Liquibase** — versioned, ordered SQL scripts (`V1_createuserstable.sql`, `V2_addphone_column.sql`, ...) that run once, in order, and are tracked in a metadata table so the same script never re-runs. This is the actual production answer to "how do I add a column safely": write a migration, review it like code, run it explicitly as part of deployment — never let the app auto-alter its own schema. Both integrate with Spring Boot as a dependency (`flyway-core` or `liquibase-core`) and run automatically on startup, migrating up to the latest version before Hibernate's `validate` check even runs.

1:36:53 — A real gotcha hit live: timezone mismatch

Running the app initially failed with a JDBC error about an invalid `timeZone` parameter — traced to a mismatch between the app's default JVM timezone and Postgres's configured timezone (`UTC`, checked via `SHOW timezone;` in pgAdmin). Fixed via a JVM argument in the IntelliJ run configuration:

```
-Duser.timezone=UTC
```

Worth knowing this exists as a category of bug — timezone-related JDBC connection failures are common enough with Postgres specifically that it's worth remembering the fix (a VM option, not a code change) the next time it happens.

 **Coming from JS?** The Docker command above is identical whether your app is Node or Java — Postgres doesn't care what's connecting to it. If you've spun up Postgres for a Node project before, this whole section is already familiar; only the `application.properties` connection config on the Spring side is new syntax for a concept (a connection string + credentials) you already know.

Part 5: Entities and relationships

1:41:52 — A first `@Entity`

```
package com.example.springdatajpademo.entities;

import jakarta.persistence.*;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
}
```

`@Entity` marks the class as mapped to a database table. `@Id` marks the primary key. `@GeneratedValue` controls how the ID is generated:

Strategy	Behavior
<code>IDENTITY</code>	Auto-increment, delegated to the database itself (used throughout this video)
<code>SEQUENCE</code>	Uses a separate database sequence object
<code>TABLE</code>	Uses a dedicated table to track the next ID (portable across DBs but slower — the video calls this out as something you generally <i>don't</i> want)
<code>AUTO</code>	Hibernate picks whichever strategy suits the underlying database
(manually assigned)	e.g. <code>UUID.randomUUID()</code> , set in code rather than delegated to the DB

Running the app with `ddl-auto=update` at this point auto-creates a `user` table — Hibernate logs the exact `CREATE TABLE` SQL it ran (because `show-sql=true`), with no SQL written by hand.

1:48:02 — `@Table`: naming collisions with reserved SQL keywords

The first run **failed** — `user` turned out to be a reserved keyword in Postgres, so Hibernate's `CREATE TABLE user (...)` was a syntax error. Fixed with:

```
@Entity
@Table(name = "users")
public class User {
    // ...
}
```

The exact same problem recurred later for the `Order` entity — `order` is *also* a reserved SQL keyword (it's part of `ORDER BY`) — fixed the same way: `@Table(name = "orders")`.

⚠️ ****Nuance** — this isn't a Postgres quirk, it's a general SQL-keyword trap worth pattern-matching on. `user`, `order`, `group`, `table`, `select` — any word that's also a SQL keyword is a landmine for entity/table naming in *any* relational database, not just Postgres. The practical habit worth building: default to `@Table(name = "...")` proactively on every entity whose class name could plausibly collide with a SQL keyword, rather than discovering it via a failed startup like the video does twice.

`@Table` also supports (mentioned, not deeply used): `schema` (which DB schema the table lives in), `uniqueConstraints`, `indexes`, and `check`-constraints.

1:53:34 — Lombok: removing getter/setter boilerplate

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
</dependency>
```

```
import lombok.Getter;
import lombok.Setter;

@Entity
@Table(name = "users")
@Getter
@Setter
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
}
```

`@Getter` / `@Setter` generate the accessor methods at compile time — identical in effect to writing them by hand, just without the visual noise. (`@AllArgsConstructor`, `@NoArgsConstructor`, and the combined `@Data` are used the same way later, for constructors.)

 **Coming from JS?** This entire annotation exists to solve a problem

JavaScript/TypeScript simply doesn't have — plain objects and classes with public fields don't need generated getter/setter methods at all. Lombok is purely working around a Java-specific convention (private fields + public accessor methods as the idiomatic way to expose state); there's nothing to port over conceptually, it's just boilerplate you'll stop noticing once Lombok is in place.

2:00:02 — `@ManyToOne` + `@JoinColumn` : a foreign-key relationship

```
@Entity
@Table(name = "orders")
@Getter
@Setter
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;
}
```

- `@ManyToOne` — many `Order` rows can belong to one `User`.
- `@JoinColumn(name = "user_id")` — names the foreign-key column that gets

created in the `orders` table, referencing `users.id`.

- IntelliJ actually *suggested* this fix automatically the moment a raw

`User user;` field was left without a relationship annotation — a live demonstration that Spring/Hibernate tooling actively flags "this looks like a relationship you forgot to declare."

Running the app now generates:

```
CREATE TABLE orders (id BIGINT ..., productname VARCHAR(255), userid BIGINT, PRIMARY KEY (id));
ALTER TABLE orders ADD CONSTRAINT ... FOREIGN KEY (user_id) REFERENCES users;
```

2:03:13 — `FetchType.LAZY` VS. `EAGER`

`EAGER` loads the related entity immediately, every time the owning entity is loaded. `LAZY` loads it only when actually accessed. The video's recommendation: use `LAZY` — better for performance, since you don't pay the cost of an extra query/join unless you actually need the related data.

⚠️ ****Nuance** — the video states "use LAZY" as a blanket rule without noting that JPA's *defaults differ by relationship type*, which matters if you ever omit `fetch` explicitly. ** Per the JPA specification: `@ManyToOne` and `@OneToOne` default to `EAGER`; `@OneToMany` and `@ManyToMany` default to `LAZY`. This video always writes `fetch = FetchType.LAZY` explicitly on its `@ManyToOne`, which is exactly the right instinct (overriding the eager default) — but if you ever see a relationship field *without* an explicit `fetch` attribute in someone else's code, don't assume it's lazy; check the annotation type against this table. Getting this wrong silently is one of the most common sources of the N+1 query problem — see below.

2:06:01 — Bidirectional relationships: mentioned, deliberately not implemented

The video shows what a `User.orders` field *would* look like:

```
@OneToMany(mappedBy = "user")
private List<Order> orders;
```

...then explicitly does **not** wire it up, citing two real reasons: performance (fetching a user would now risk pulling every one of their orders along with it) and a **circular dependency** risk when serializing to JSON (`User` → `orders` → each `Order.user` → its `orders` → ... — infinite recursion). The fix is named but deferred: DTOs (already the pattern this whole video follows) or `@JsonIgnore` on one side of the relationship.

📌 **Beyond the lecture — auto-increment `Long` vs. `UUID` primary keys is a real production decision, not just an artifact of the in-memory demo.** This video uses `IDENTITY` (auto-increment `Long`) once a real database is involved, purely because that's the natural default for a single Postgres instance. In practice this is an actual architectural fork worth knowing about *before* you need it:

	Auto-increment <code>Long</code>	<code>UUID</code>
Human-readable, sequential	Yes	No
Predictable/guessable (a security concern for public-facing IDs)	Yes — bad for exposing e.g. <code>/orders/42</code> , <code>/orders/43</code>	No
Safe to generate before talking to the database	No — DB assigns it	Yes — client or app code can generate it up front
Works cleanly across multiple databases/services (no collision risk)	No — two DBs can both mint <code>id = 1</code>	Yes
Index/storage size, insert performance at very high write volume	Better	Slightly worse (larger, less sequential — can fragment indexes)

The "safe across multiple databases" row is exactly why UUIDs are the default choice once you reach **microservices** (video 4's topic) — if each service owns its own database, auto-increment IDs from two different services can collide the moment data needs to merge or reference across services. Nothing to act on yet; just recognize this choice will come back with real stakes later in the series.

Part 6: `JpaRepository`, entity lifecycle, and the persistence context

2:15:00 — `JpaRepository<Entity, IdType>`

```
public interface UserRepository extends JpaRepository<User, Long> {  
    }  
}
```

That's the entire interface. `save()`, `findAll()`, `findById()`, `deleteById()`, and more all come free — no implementation written, because Spring Data JPA generates one at runtime. The video opens `SimpleJpaRepository` (Spring Data's actual implementation) in the IDE to show that under the hood, `save()` ultimately calls `EntityManager.persist(...)` — connecting straight back to the JPA layer discussed in Part 3.

■ ****Coming from JS? — this is the one concept in this video with no clean Node ORM parallel for most people, so it's worth sitting with.**** Prisma and Sequelize don't do this — you always call an explicit `.update()` / `.save()` yourself; there's no ambient "if I mutate a fetched object, it silently persists later" behavior. **TypeORM is the exception** — because it was built as a deliberate JPA/Hibernate port, it has the exact same dirty-checking behavior inside a transaction. If you've only used Prisma/Sequelize/Mongoose, budget real time to internalize this — it's easy to be surprised the first time an update "just happens" with no `.save()` call anywhere near it, and equally easy to be surprised the *other* direction when you forget `@Transactional` and a mutation silently does nothing.

⚠ ****Nuance — `@Transactional` here is doing double duty, and the video only shows one side of it.**** `@Transactional` is what makes dirty checking *possible* (an active persistence context requires an active transaction) — but its primary, more fundamental job is wrapping the whole method in a database transaction: if an exception is thrown partway through a `@Transactional` method, **every change made so far in that method rolls back**. That matters enormously the moment a method does more than one write (e.g. "create an order, then decrement stock" — if decrementing stock fails, you want the order creation undone too, not left half-committed). This video's methods only ever do one logical write each, so the rollback behavior never gets exercised on screen — but it's the reason `@Transactional` exists at all, dirty checking is really a side effect of it.

Part 7: Full CRUD against the real database

The same five operations from Part 2 are rebuilt, now against Postgres:

Create user

```
@PostMapping
public ResponseEntity<UserDTO> createUser(@RequestBody CreateUserDTO createUserDTO) {
    return
    ResponseEntity.status(HttpStatus.CREATED).body(userService.createUser(createUserDTO));
}
```

```
public UserDTO createUser(CreateUserDTO createUserDTO) {
    User user = new User();
    user.setName(createUserDTO.getName());
    user.setEmail(createUserDTO.getEmail());
    User savedUser = userRepository.save(user);
    return new UserDTO(savedUser.getId(), savedUser.getName(), savedUser.getEmail());
}
```

Get all users / get user by ID

```
public List<UserDTO> getAllUsers() {
    List<User> users = userRepository.findAll();
    List<UserDTO> userDTOList = new ArrayList<>();
    for (User user : users) {
        userDTOList.add(new UserDTO(user.getId(), user.getName(), user.getEmail()));
    }
    return userDTOList;
}

public UserDTO getUserById(Long id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("User not found"));
    return new UserDTO(user.getId(), user.getName(), user.getEmail());
}
```

This is the first place `Optional` is actually handled (`findById` returns `Optional<User>`), via `.orElseThrow(...)` — exactly the pattern the earlier, in-memory version explicitly deferred.

⚠️ **Nuance** — `orElseThrow(() -> new RuntimeException(...))` is a deliberate placeholder, not the finished error-handling story. The video says so directly: "we'll throw the proper error there... we haven't learned exception handling yet, we'll see it in a future video." A plain `RuntimeException` reaching the client today produces an undifferentiated `500 Internal Server Error` with no useful message — not the `404 Not Found` a missing user should actually return. This is exactly what video 1's notes flagged as a gap and gave a fix for: a custom exception (`UserNotFoundException`) plus a `@RestControllerAdvice` + `@ExceptionHandler` (see [video 1 → Building this into a scalable, robust backend](#)). Wiring that in turns every `orElseThrow(() -> new RuntimeException(...))` in this video's code into a proper `404` with a clear message, with "one change in one place" rather than a `try / catch` in every controller method.

Update user — relying on dirty checking

Shown in full in Part 6 above.

Patch user — conditional partial update, same dirty-checking pattern

```
@Transactional

public UserDTO patchUser(Long id, PatchUserDTO patchUserDTO) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("User not found"));

    if (patchUserDTO.getName() != null) {
        user.setName(patchUserDTO.getName());
    }
    if (patchUserDTO.getEmail() != null) {
        user.setEmail(patchUserDTO.getEmail());
    }

    // again, no explicit save() — dirty checking handles it

    return new UserDTO(user.getId(), user.getName(), user.getEmail());
}
```

The distinction from `updateUser` above: only the fields actually present in the request body get touched — a client can send just `{"name": "..."}` and leave `email` untouched, which is the whole point of `PATCH` vs. `PUT`.

Delete user

```
public void deleteUserById(Long id) {
    userRepository.deleteById(id);
}
```

One line — `deleteById` is one of `JpaRepository`'s free methods.

2:59:00 — Nested resource: orders belong to a user

The video makes an explicit, named design decision here, worth reading as the single clearest "this is the correct, standard way" statement in the video:

A `GET / POST /orders` on its own would be wrong: an order can't exist without an owning user — so the resource path should reflect that ownership directly.

```
@RestController
@RequestMapping("/api/v1/users/{userId}/orders")
public class OrderController {
    // POST    /api/v1/users/{userId}/orders    → create an order for this user
    // GET     /api/v1/users/{userId}/orders    → list this user's orders
}
```

```
@PostMapping
public ResponseEntity<OrderDTO> createOrder(
    @PathVariable Long userId,
    @RequestBody CreateOrderDTO createOrderDTO) {
    return ResponseEntity.status(HttpStatus.CREATED).body(orderService.createOrder(userId,
createOrderDTO));
}
```

```
public OrderDTO createOrder(Long userId, CreateOrderDTO createOrderDTO) {
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new RuntimeException("User not found"));

    Order order = new Order();
    order.setUser(user);
    order.setProductName(createOrderDTO.getProductName());

    Order savedOrder = orderRepository.save(order);

    return new OrderDTO(savedOrder.getId(), savedOrder.getProductName(), new
UserDTO(user.getId(), user.getName(), user.getEmail()));
}
```

Fetching a user's orders is where **derived query methods** first appear in this video (full explanation in Part 8):

```
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByUserId(Long userId);
}
```

No implementation, no `@Query` — Spring Data JPA parses the method name (`findByUserId`) and generates `SELECT * FROM orders WHERE user_id = ?` entirely on its own.

A live gotcha: `LazyInitializationException` when serializing a relationship

While testing `GET /users/{userId}/orders`, the response included an unexpected extra field:

`hibernateLazyInitializer` — a symptom of trying to serialize a lazily-loaded relationship (`Order.user`, `FetchType.LAZY`) directly to JSON. The video names the cause (the `@ManyToOne(fetch = FetchType.LAZY)` from Part 5) and the fix in principle (map the entity to a DTO before returning it, which the code above already does correctly by converting `user` → `new UserDTO(...)` rather than returning the `User` entity directly) — but doesn't stop to explain the mechanism in depth.

📌 ****Beyond the lecture — this is one of the most common real-world Spring**

Data JPA bugs, worth understanding properly since the video hits it live and moves past it quickly.**

`LazyInitializationException` (or the Jackson-serialization symptom shown here) happens when code tries to access a `LAZY` relationship **after** the Hibernate session/transaction that fetched the owning entity has already closed — there's no active persistence context left to go fetch the related row on demand. Two real fixes, both better than the video's workaround of just not touching the lazy field carelessly:

1. ****Never serialize entities directly — always map to a DTO inside the**

`@Transactional` /repository-backed service method**, exactly as this video's code already does. This is the single most important habit for avoiding the whole class of bug: entities should never leave the service layer.

1. **When you *do* need the related data, fetch it explicitly — either**

`@EntityGraph(attributePaths = "user")` on the repository method, or a JPQL `JOIN FETCH` (see Part 8) — rather than relying on lazy-loading to "just work" outside its valid window.

The so-called **"Open Session in View" (OSIV)** Spring Boot default (enabled unless you explicitly disable it via `spring.jpa.open-in-view=false`) keeps the Hibernate session open for the entire HTTP request specifically to paper over this problem — most guides recommend disabling it and fixing fetching explicitly instead, since OSIV hides N+1 query problems (see below) rather than solving them.

📌 **Beyond the lecture — the N+1 query problem, never named in this video but directly set up by it.**

Picture `GET /users` returning 100 users, then some code loops over them accessing `user.getOrders()` for each one (the bidirectional `@OneToMany` the video mentioned but didn't wire up). With lazy loading, that's **1 query to fetch the users, plus 1 additional query per user** to fetch their orders — 101 queries total for what should be a handful. This is the classic **N+1 query problem**, one of the most common Spring Data JPA/Hibernate performance bugs in real applications. The fix: a JPQL `JOIN FETCH` (below) or `@EntityGraph` to pull the related data in a single query up front, instead of one query per related row. Worth watching for the moment you *do* wire up a bidirectional relationship and loop over a collection touching a lazy field inside that loop.

Part 8: Spring Data JPA query techniques

3:24:17 — Derived query methods (method-name query generation)

```
List<User> findByName(String name);
List<User> findByNameAndEmail(String name, String email);
List<User> findByNameContaining(String keyword);
```

Spring Data JPA parses the method name itself and generates the corresponding SQL — no query written anywhere. ****Limitation, stated directly:**** this can't express genuinely complex queries, and method names get unwieldy long once you're combining several conditions.

3:26:52 — `@Query` (JPQL) — custom queries when method names aren't enough

```
public interface UserRepository extends JpaRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.name = :name")
    List<User> findUsersByCustomQuery(@Param("name") String name);

}
```

The key distinction, stated directly: **JPQL operates on entities and their fields** (`User` , `u.name`) — not **on tables and columns** (`users` , `name`). `SELECT u FROM User u` — not `SELECT * FROM users`.

`@Param("name")` binds the method's `name` parameter to the `:name` placeholder in the query string.

3:26:37 — Native queries — raw SQL, when even JPQL isn't enough

```
@Query(value = "SELECT * FROM users WHERE name = ?1", nativeQuery = true)
List<User> findUsersNative(String name);
```

Used when you need genuine, complex joins, database-specific features, or fine-grained performance tuning that JPQL can't express. **Downside, stated directly:** the query becomes tied to your specific database's SQL dialect — no longer portable.

 **Beyond the lecture** — parameterized placeholders (`?1` , `:name`) here aren't just style, they're what prevents SQL injection. Every query in this video — derived, JPQL, and native — uses a parameter placeholder that Spring Data JPA binds safely, rather than string-concatenating user input directly into the query text. This is the correct, safe pattern by default. The moment you (or a future teammate) build a query by concatenating a `String` request parameter directly into a `@Query(value = "...")` string, you've reintroduced classic SQL injection — Spring Data JPA doesn't stop you from writing that, it just never demonstrates it. Treat "always use `?1` / `:name` placeholders, never string-concatenate into a query" as a hard rule, not a style preference.

3:27:39 — `@Modifying` — required for UPDATE/DELETE queries

```
@Modifying
@Transactional
@Query("UPDATE User u SET u.name = :name WHERE u.id = :id")
void updateUserName(@Param("id") Long id, @Param("name") String name);
```

JPQL `@Query` methods default to expecting a `SELECT`-style result. Any query that actually modifies data must be marked `@Modifying`, and — same reasoning as Part 6 — needs `@Transactional`, since it's a write.

3:28:13 — Pagination and sorting

The stated motivation: fetching "a million users" in one response isn't feasible — real applications page through large collections instead.

```
@GetMapping("/paginated")
public ResponseEntity<List<UserDTO>> getUsersPaginated(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int pageSize,
    @RequestParam(defaultValue = "ASC") String direction,
    @RequestParam(defaultValue = "name") String sortBy) {
    return ResponseEntity.ok(userService.getUsersPaginated(page, pageSize, direction, sortBy));
}
```

```
public List<UserDTO> getUsersPaginated(int page, int pageSize, String direction, String sortBy)
{
    Sort sort = direction.equalsIgnoreCase("ASC")
        ? Sort.by(sortBy).ascending()
        : Sort.by(sortBy).descending();

    Pageable pageable = PageRequest.of(page, pageSize, sort);
    Page<User> usersPage = userRepository.findAll(pageable);

    List<UserDTO> userDTOList = new ArrayList<>();
    usersPage.forEach(user -> userDTOList.add(new UserDTO(user.getId(), user.getName(),
user.getEmail())));
    return userDTOList;
}
```

- `Pageable` — an interface describing *which page, how big, sorted how*.
- `PageRequest.of(page, size, sort)` — the concrete implementation.
- `Sort.by(field).ascending()` / `.descending()` — the sort direction.
- `userRepository.findAll(pageable)` — `JpaRepository` already overloads

`findAll()` to accept a `Pageable` and return a `Page<User>` — no extra code needed on the repository interface at all.

Tested live with `?page=0&pageSize=2&direction=DESC&sortBy=id` against 10 seeded users, confirming page boundaries and sort order both work exactly as configured — including that

`@RequestParam(defaultValue = ...)` correctly falls back when a client omits a parameter entirely.

Coming from JS?

Spring Data JPA	Node/SQL equivalent
<code>findByName(String name)</code> (derived query)	An ORM's built-in filter, e.g. Prisma's <code>prisma.user.findMany({ where: { name } })</code>
<code>@Query</code> (JPQL)	Prisma's typed query builder / Sequelize's <code>Model.findAll({...})</code> — still object/model-shaped, not raw SQL
<code>@Query(nativeQuery = true)</code>	Prisma's <code>\$queryRaw</code> , Sequelize's <code>sequelize.query(...)</code> , or a raw <code>pg</code> query — an escape hatch to real SQL
<code>Pageable</code> / <code>PageRequest.of(page, size, sort)</code>	Manually building <code>LIMIT ? OFFSET ? + ORDER BY</code> , or Prisma's <code>skip</code> / <code>take</code> / <code>orderBy</code>

The pattern across both ecosystems is identical: model-shaped query methods for the common cases, an explicit raw-SQL escape hatch for when they're not enough, and pagination as parameters bolted onto whichever layer you're using.

Beyond the lecture — filling the gaps this video leaves open

A few things this video's code needs before it's production-ready, since none of them were covered here (some are named as deferred to a future video in the series; others aren't mentioned at all):

- **Input validation.** Nothing in this video validates `CreateUserDTO` or

`CreateOrderDTO` — an empty `name` or malformed `email` is accepted as-is. The fix is Bean Validation, referenced in [video 1's roadmap](#), now concretely applicable:

```
public class CreateUserDTO {          @NotBlank(message = "name is required")          private
String name;

    @Email(message = "must be a valid email")          private String email; }
```

```
@PostMapping      public ResponseEntity<UserDTO> createUser(@Valid @RequestBody CreateUserDTO
dto) {              // @Valid triggers validation; a violation throws MethodArgumentNotValidException
// before this method body ever runs      }
```

- **Proper error handling for "not found."** Flagged repeatedly by the video

itself as deferred — see the nuance box under "Get all users / get user by ID" above.

`@RestControllerAdvice` + a custom `UserNotFoundException` (already sketched in video 1's notes) turns every `orElseThrow(() -> new RuntimeException(...))` in this codebase into a real `404`.

- ****Entity ↔ DTO mapping** is entirely hand-written here, repeated in every

service method.** Every `new UserDTO(user.getId(), user.getName(), user.getEmail())` is boilerplate that scales badly as entities grow more fields. **MapStruct** is the standard production answer — an annotation processor that generates mapping code at compile time from an interface you declare:

```
@Mapper(componentModel = "spring") public interface UserMapper {
    UserDTO toDto(User user);
    User toEntity(CreateUserDTO dto);
}
```

Inject `UserMapper` like any other bean; call `userMapper.toDto(user)` instead of hand-rolling the constructor call everywhere.

- **`spring.jpa.hibernate.ddl-auto=update` on a project with a real

team.** The moment more than one person touches this schema, `update`'s "just wing it" approach to schema changes stops being safe — see the Flyway/Liquibase note under Part 4.

- **The N+1 query problem and `LazyInitializationException`** — both

covered in detail in Part 7's notes above, since the video hits the latter live but doesn't explain the mechanism.

- **This is exactly where video 1's `@Profile` / `@ConditionalOnProperty`

concepts become concretely useful.** Every `application.properties` value in this video (`spring.datasource.url`, username, password) is exactly the kind of thing that should differ between `application-dev.properties` and `application-prod.properties`, switched via `spring.profiles.active` — the video hardcodes one `application.properties` throughout, which is fine for learning, not for a real deployment.

- **Connection pooling, recapped.** Video 1 noted Spring Boot defaults to

HikariCP with a small connection pool. Now that a real `DataSource` is actually configured, that pool is genuinely in play — every request that touches `userRepository` borrows a connection from it. Under real load, pool size needs tuning alongside expected concurrent request volume (see video 1's threading-model section for why).

- **Testing the repository layer isn't covered.** The video verifies every

endpoint by hand through Postman and pgAdmin. The standard automated equivalent: `@DataJpaTest` (an in-memory or Testcontainers-backed slice test that boots just the JPA layer, not the whole app) — worth reaching for once manual Postman-clicking starts feeling repetitive.

- **API versioning (`/api/v1/...`) is used throughout without ever being

explained.** The `v1` segment is a deliberate, common REST convention: it lets you introduce a breaking `/api/v2/...` later without breaking existing clients still calling `v1`. Worth keeping in any API you build, even a small one — retrofitting versioning after clients already depend on unversioned URLs is far more painful.

Interview and practical takeaways

1. **The four HTTP methods map to CRUD, and resource paths are always

plural** — `/users` (collection), `/users/{id}` (single resource); `PUT` replaces the whole resource, `PATCH` updates part of it.

1. **DTOs are not entities.** Entities mirror database tables; DTOs are the

API's actual request/response contract. Never return an entity directly from a controller — it's both an information-leak risk (exposes internal fields/relationships) and the direct cause of

`LazyInitializationException`.

1. ****Know the JDBC → Hibernate → JPA → Spring Data JPA lineage and what**

each one specifically fixed** — a very common "why do we even need an ORM" / "what's the difference between Hibernate and JPA" interview question maps directly onto this progression.

1. ****Constructor injection + `@Transactional` + dirty checking is the**

idiomatic Spring Data JPA update pattern** — fetch a managed entity, mutate its fields, let Hibernate detect and persist the change. No explicit `.save()` call needed on update, only on create.

1. **`ddl-auto` values, and which ones are production-safe —**

`validate` / `none` in production, paired with Flyway/Liquibase for actual schema changes; `create` / `create-drop` / `update` are for local development only.

1. ****Derived query methods → JPQL → native queries is an escalation**

ladder**, not three unrelated features — reach for the simplest one that expresses the query; drop to native SQL only when JPQL genuinely can't.

1. **`@ManyToOne` / `@JoinColumn` is how a foreign key is expressed in JPA,**

and default fetch types differ by relationship type (`@ManyToOne` / `@OneToOne` eager,

`@OneToMany` / `@ManyToMany` lazy) — a detail worth stating precisely rather than "lazy is just better," since getting it wrong silently causes the N+1 query problem.

Next: Video 003 — Security in Spring Boot.